

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Technical Presentation Questions

Course Code:	CSA0315	Course Title:	Data Structures
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Technical Presentations Questions
Weightage:	10% of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	HARSHITH G	Reg. No.:	192525394
Section / Batch:	B TECH – AIML	Date:	15/07/2026

Code of Conduct

I HARSHITH G / 192525394 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARSHITH G

Instructions

- This test contains 3 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should contain following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- Create a PPT with 15 slides – Laptop permitted for presentation in the class. Take Print out and submit in the class.

Section / Topic	Focus	Q Nos	Marks
Realization of ADT in 'C'- Array DS- Performance analysis and Measurement: Time and space analysis of algorithms.	Linked List data Structure	1	30
Primitive and non-primitive data structure, Linear and non-linear data structure Arrays	Tries data Structure	2	40
Dynamic linear Data Structures	Queue Data structures	3	40
GRAND TOTAL:			100 Marks

Annexure A – Technical Presentation

1) A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation

2) A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.

3) A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

TECHNICAL PRESENTATION

CSA0315 – DATA STRUCTURES

Faculty:

Dr T Rajesh Kumar

By :

Harshith G (192525394)

1) MUSIC STREAMING PLAYLIST MANAGEMENT

Music streaming applications such as Spotify, Apple Music, and YouTube Music allow users to create and modify playlists. Songs are frequently added, removed, and reordered based on user preferences. Therefore, the application requires a data structure that supports dynamic updates efficiently without affecting performance.

- **Key Points**
- Playlists change dynamically.
- Users can add or remove songs anytime.
- Songs can be reordered easily.
- Efficient data management improves user experience.

PROBLEM STATEMENT

A playlist management system should support the following operations efficiently:

- Insert new songs at any position.
 - Delete songs from the playlist.
 - Move to the next or previous song.
 - Rearrange songs without shifting all elements.
 - Handle playlists of varying sizes dynamically.
-
- **Challenge:** Arrays require shifting elements after insertion or deletion, making them less efficient for dynamic playlists.



RECOMMENDATION

A Doubly Linked List consists of nodes where each node contains:

- Song information
 - Pointer to the previous song
 - Pointer to the next song
 - This structure enables easy navigation in both forward and backward directions while allowing quick insertion and deletion of songs.
-
- **Why Doubly Linked List?**
 - Dynamic memory allocation
 - Efficient insertion and deletion
 - Supports previous and next song navigation
 - No need to shift existing elements

PERFORMANCE ANALYSIS

OPERATION	TIME COMPLEXITY
Insert Song	$O(1)$
Delete Song	$O(1)$
Next Song	$O(1)$
Previous Song	$O(1)$
Search Song	$O(n)$

Advantages

- ✓ Flexible playlist size
- ✓ Fast updates
- ✓ Easy song reordering
- ✓ Better than arrays for dynamic data

2) DICTIONARY WORD SEARCH SYSTEM

Dictionary applications contain thousands or even millions of words. Users expect search results instantly when they enter a word. Therefore, selecting an efficient data structure is essential for achieving fast lookup performance.

- **Key Points**
- Large number of stored words
- Instant search required
- Efficient insertion and deletion
- High-performance lookup system

PROBLEM STATEMENT

The dictionary application should perform the following tasks:

- Search whether a word exists.
- Insert new words.
- Delete unwanted words.
- Handle a large database efficiently.
- Reduce search time.
- Searching every word one by one becomes slow as the database grows larger.



RECOMMENDATION

- A Tree is a tree-based data structure where each node represents a character of a word.
- Words that share the same prefix also share the same path in the tree, reducing redundant storage.
- **Why Tree?**
- Fast word searching.
- Efficient prefix matching.
- Shared prefixes save memory.
- Ideal for dictionaries and autocomplete systems.

PERFORMANCE ANALYSIS

Operation	Time Complexity
Search Word	$O(L)$
Insert Word	$O(L)$
Delete Word	$O(L)$
Prefix Search	$O(L)$

Where L = Length of the word

Advantages

- ✓ Very fast string lookup.
- ✓ Excellent for autocomplete.
- ✓ Efficient prefix searching.
- ✓ Widely used in search engines and dictionaries.

3) TASK SCHEDULING SYSTEM

Task scheduling systems are used in operating systems, project management software, and workflow applications to organize and execute tasks. Each task is assigned a priority level, and the scheduler selects tasks based on these priorities. A Binary Search Tree (BST) helps maintain tasks in sorted order according to their priority.

- **Key Points**
- Organizes tasks based on priority.
- Supports dynamic insertion and deletion.
- Maintains tasks in sorted order.
- Suitable for scheduling applications.

PROBLEM STATEMENT

A task scheduling system should efficiently perform the following operations:

- Add new tasks with different priorities.
- Delete completed tasks.
- Retrieve tasks in priority order.
- Handle a changing number of tasks.
- Maintain efficient scheduling performance.
- A simple queue cannot prioritize important tasks, making it unsuitable for priority-based scheduling.



RECOMMENDATIONS

A **Binary Search Tree (BST)** is a hierarchical data structure where:

- Each node stores a task and its priority.
 - Left subtree contains lower-priority tasks.
 - Right subtree contains higher-priority tasks.
 - This structure keeps tasks sorted automatically, making it easier to retrieve tasks according to priority.
-
- **Why BST?**
 - Maintains tasks in sorted order.
 - Efficient insertion and deletion.
 - Easy retrieval of highest or lowest priority task.
 - Supports dynamic scheduling.

PERFORMANCE ANALYSIS

Operation	Time Complexity
Insert Task	$O(\log n)$ (Average)
Search Task	$O(\log n)$ (Average)
Delete Task	$O(\log n)$ (Average)
Retrieve Highest Priority	$O(\log n)$

Advantages

- ✓ Keeps tasks sorted automatically.
- ✓ Handles dynamic task updates.
- ✓ Efficient searching and deletion.
- ✓ Suitable for applications requiring ordered processing.

CONCLUSION

- Selecting the appropriate data structure is essential for improving the efficiency, speed, and flexibility of software applications.
- Doubly Linked List is ideal for music streaming playlists because it supports efficient insertion, deletion, and navigation between songs.
- Tree (Prefix Tree) is best suited for dictionary applications as it provides fast word searching, prefix matching, and autocomplete functionality.
- Binary Search Tree (BST) is suitable for task scheduling systems because it maintains tasks in sorted priority order and supports efficient insertion, deletion, and searching.
- Overall, choosing the right data structure improves application performance, reduces execution time, optimizes memory usage, and enhances the overall user experience.



Thank You